



```
In [ ]: import numpy as np
import pandas as pd
import torch
import matplotlib.pyplot as plt
import seaborn as sns

data = pd.read_csv('California_Houses.csv')
X = data.drop('Median_House_Value', axis=1)
y = data['Median_House_Value']

def scaler(X):
    mean = np.mean(X, axis=0)
    std = np.std(X, axis=0)
    std[std == 0] = 1
    return (X - mean) / std, mean, std

def train_test(X, y, test_size=0.2, random_state=42):
    np.random.seed(random_state)
    idx = np.arange(len(X))
    np.random.shuffle(idx)

    test_data = int(len(X) * test_size)
    test_idx = idx[:test_data]
    train_idx = idx[test_data:]

    return X[train_idx], X[test_idx], y[train_idx], y[test_idx]

X_scaled, X_mean, X_std = scaler(X.values)
y_scaled, y_mean, y_std = scaler(y.values.reshape(-1, 1))
X_train, X_test, y_train, y_test = train_test(X_scaled, y_scaled)

X_train_tensor = torch.FloatTensor(X_train)
y_train_tensor = torch.FloatTensor(y_train)
X_test_tensor = torch.FloatTensor(X_test)
y_test_tensor = torch.FloatTensor(y_test)

class LinearRegression:
    def __init__(self, n_features, learning_rate=0.01):
        self.w = torch.randn(n_features, 1, requires_grad=False)
        self.b = torch.randn(1, requires_grad=False)
        self.learning_rate = learning_rate
        self.loss_history = []
        self.rmse_history = []

    def predict(self, X):
        return X @ self.w + self.b

    def mse_loss(self, y_pred, y_true):
        return torch.mean((y_pred - y_true) ** 2)
```

```

def rmse(self, y_pred, y_true):
    return torch.sqrt(torch.mean((y_pred - y_true) ** 2))

def fit(self, X, y, n_epochs=1000, verbose=True):
    n_samples = X.shape[0]

    for epoch in range(n_epochs):

        y_pred = self.predict(X)
        error = y_pred - y
        changes=(2*(X.t() @ error))/n_samples

        self.w -= self.learning_rate * changes

        loss = self.mse_loss(y_pred, y)
        rmse_val = self.rmse(y_pred, y)
        self.loss_history.append(loss.item())
        self.rmse_history.append(rmse_val.item())
        if epoch % 100 == 0:
            print(f'Epoch {epoch}: MSE = {loss.item():.6f}, RMSE = {rmse_v

def evaluate(self, X, y):
    y_pred = self.predict(X)
    mse = self.mse_loss(y_pred, y).item()
    rmse_val = self.rmse(y_pred, y).item()
    mae = torch.mean(torch.abs(y_pred - y)).item()

    return {
        'mse': mse,
        'rmse': rmse_val,
        'mae': mae
    }

model = LinearRegression(n_features=X_train_tensor.shape[1], learning_rate=0.01)
model.fit(X_train_tensor, y_train_tensor, n_epochs=1000)

train_metrics = model.evaluate(X_train_tensor, y_train_tensor)
test_metrics = model.evaluate(X_test_tensor, y_test_tensor)

print("\n Результаты линейной регрессии ")
print("Train Metrics:")
print(f"MSE: {train_metrics['mse']:.6f}, RMSE: {train_metrics['rmse']:.6f}, MAE: {train_metrics['mae']:.6f}")
print("Test Metrics:")
print(f"MSE: {test_metrics['mse']:.6f}, RMSE: {test_metrics['rmse']:.6f}, MAE: {test_metrics['mae']:.6f}")

# Графики
plt.figure(figsize=(15, 5))

plt.subplot(1, 2, 1)
plt.plot(model.loss_history)
plt.title('MSE Loss during Training')
plt.xlabel('Epoch')

```

```
plt.ylabel('MSE')
plt.grid(True)

plt.subplot(1, 2, 2)
plt.plot(model.rmse_history)
plt.title('RMSE during Training')
plt.xlabel('Epoch')
plt.ylabel('RMSE')
plt.grid(True)

plt.tight_layout()
plt.show()
```

Epoch 0: MSE = 8.325172, RMSE = 2.885338
 Epoch 100: MSE = 1.288124, RMSE = 1.134956
 Epoch 200: MSE = 1.180092, RMSE = 1.086321
 Epoch 300: MSE = 1.133311, RMSE = 1.064571
 Epoch 400: MSE = 1.107355, RMSE = 1.052310
 Epoch 500: MSE = 1.090684, RMSE = 1.044358
 Epoch 600: MSE = 1.078967, RMSE = 1.038733
 Epoch 700: MSE = 1.070292, RMSE = 1.034549
 Epoch 800: MSE = 1.063680, RMSE = 1.031349
 Epoch 900: MSE = 1.058559, RMSE = 1.028863

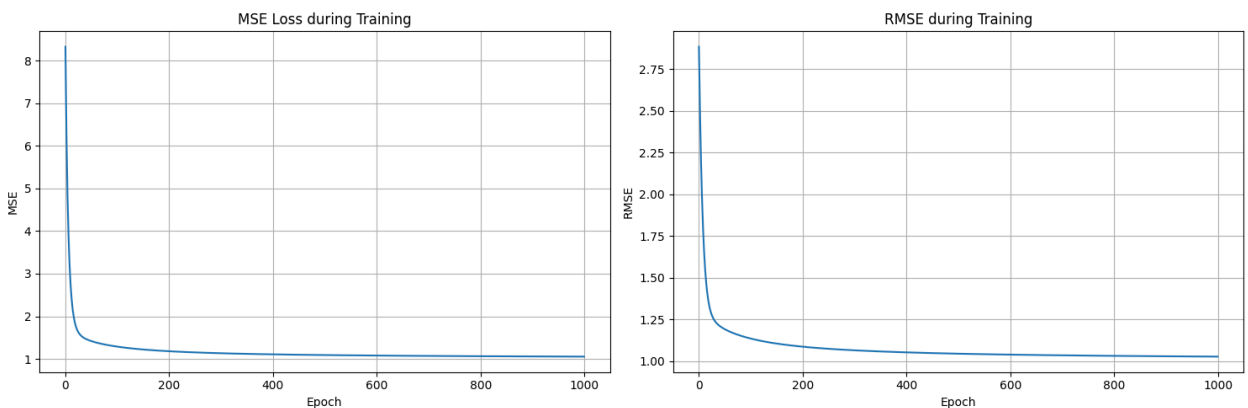
Результаты линейной регрессии

Train Metrics:

MSE: 1.054554, RMSE: 1.026915, MAE: 0.843882

Test Metrics:

MSE: 1.072748, RMSE: 1.035736, MAE: 0.849730



```
In [ ]: import torch
import torch.nn as nn
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, f1_score

data = pd.read_csv('Titanic-Dataset.csv')
```

```

features = ['Pclass', 'Sex', 'Age', 'SibSp', 'Parch', 'Fare']
target = 'Survived'

data['Sex'] = data['Sex'].map({'male': 0, 'female': 1})
data['Age'].fillna(data['Age'].median(), inplace=True)
data['Fare'].fillna(data['Fare'].median(), inplace=True)

X = data[features].values
y = data[target].values

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

X_train_tensor = torch.FloatTensor(X_train_scaled)
y_train_tensor = torch.FloatTensor(y_train).reshape(-1, 1)
X_test_tensor = torch.FloatTensor(X_test_scaled)
y_test_tensor = torch.FloatTensor(y_test).reshape(-1, 1)

class LogisticRegressionTorch:
    def __init__(self, input_dim, learning_rate=0.01, n_epochs=1000):
        self.learning_rate = learning_rate
        self.n_epochs = n_epochs
        self.weights = torch.randn(input_dim, 1, requires_grad=True)
        self.bias = torch.randn(1, requires_grad=True)
        self.loss_history = []
        self.accuracy_history = []
        self.f1_history = []

    def sigmoid(self, z):
        return 1 / (1 + torch.exp(-z))

    def forward(self, X):
        linear = X@self.weights + self.bias
        return self.sigmoid(linear)

    def compute_loss(self, y_pred, y_true):
        loss = -torch.mean(y_true * torch.log(y_pred + 1e-8) +
                           (1 - y_true) * torch.log(1 - y_pred + 1e-8))
        return loss

    def predict_proba(self, X):
        with torch.no_grad():
            return self.forward(X)

    def predict(self, X, threshold=0.5):
        proba = self.predict_proba(X)
        return (proba >= threshold).float()

    def compute_metrics(self, X, y):

```

```

        y_pred = self.predict(X)
        y_true = y
        accuracy = accuracy_score(y_true.numpy(), y_pred.numpy())
        f1 = f1_score(y_true.numpy(), y_pred.numpy(), zero_division=0)

    return accuracy, f1

def fit(self, X, y):
    for epoch in range(self.n_epochs):
        y_pred = self.forward(X)
        loss = self.compute_loss(y_pred, y)
        loss.backward()
        with torch.no_grad():
            self.weights -= self.learning_rate * self.weights.grad
            self.bias -= self.learning_rate * self.bias.grad
            self.weights.grad.zero_()
            self.bias.grad.zero_()

        if epoch % 10 == 0:
            self.loss_history.append(loss.item())
            train_acc, train_f1 = self.compute_metrics(X, y)
            self.accuracy_history.append(train_acc)
            self.f1_history.append(train_f1)

            if epoch % 100 == 0:
                print(f'Epoch {epoch}, Loss: {loss.item():.4f}, '
                      f'Accuracy: {train_acc:.4f}, F1: {train_f1:.4f}')

input_dim = X_train_tensor.shape[1]
model = LogisticRegressionTorch(input_dim, learning_rate=0.1, n_epochs=1000)
model.fit(X_train_tensor, y_train_tensor)
y_test_pred = model.predict(X_test_tensor)
test_accuracy = accuracy_score(y_test, y_test_pred.numpy())
test_f1 = f1_score(y_test, y_test_pred.numpy())

print(f"\nFinal Test Results:")
print(f"Accuracy: {test_accuracy:.4f}")
print(f"F1 Score: {test_f1:.4f}")

# графики
fig, (ax1, ax2, ax3) = plt.subplots(3, 1, figsize=(5, 15))

ax1.plot(model.loss_history)
ax1.set_title('Training Loss Over Epochs')
ax1.set_xlabel('Epochs (every 10th)')
ax1.set_ylabel('Loss')
ax1.grid(True)

ax2.plot(model.accuracy_history)
ax2.set_title('Training Accuracy Over Epochs')
ax2.set_xlabel('Epochs (every 10th)')
ax2.set_ylabel('Accuracy')
ax2.grid(True)

```

```
ax3.plot(model.fl_history)
ax3.set_title('Training F1 Score Over Epochs')
ax3.set_xlabel('Epochs (every 10th)')
ax3.set_ylabel('F1 Score')
ax3.grid(True)

print("\nModel Weights:")
for i, feature in enumerate(features):
    print(f"{feature}: {model.weights[i].item():.4f}")
print(f"Bias: {model.bias.item():.4f}")
```

Epoch 0, Loss: 1.1348, Accuracy: 0.5421, F1: 0.5248
Epoch 100, Loss: 0.4630, Accuracy: 0.7949, F1: 0.7355
Epoch 200, Loss: 0.4413, Accuracy: 0.7879, F1: 0.7219
Epoch 300, Loss: 0.4399, Accuracy: 0.7907, F1: 0.7246
Epoch 400, Loss: 0.4398, Accuracy: 0.7907, F1: 0.7236
Epoch 500, Loss: 0.4397, Accuracy: 0.7935, F1: 0.7273
Epoch 600, Loss: 0.4397, Accuracy: 0.7935, F1: 0.7273
Epoch 700, Loss: 0.4397, Accuracy: 0.7935, F1: 0.7273
Epoch 800, Loss: 0.4397, Accuracy: 0.7935, F1: 0.7273
Epoch 900, Loss: 0.4397, Accuracy: 0.7935, F1: 0.7273

Final Test Results:

Accuracy: 0.8045

F1 Score: 0.7368

Model Weights:

Pclass: -0.9156

Sex: 1.3160

Age: -0.5151

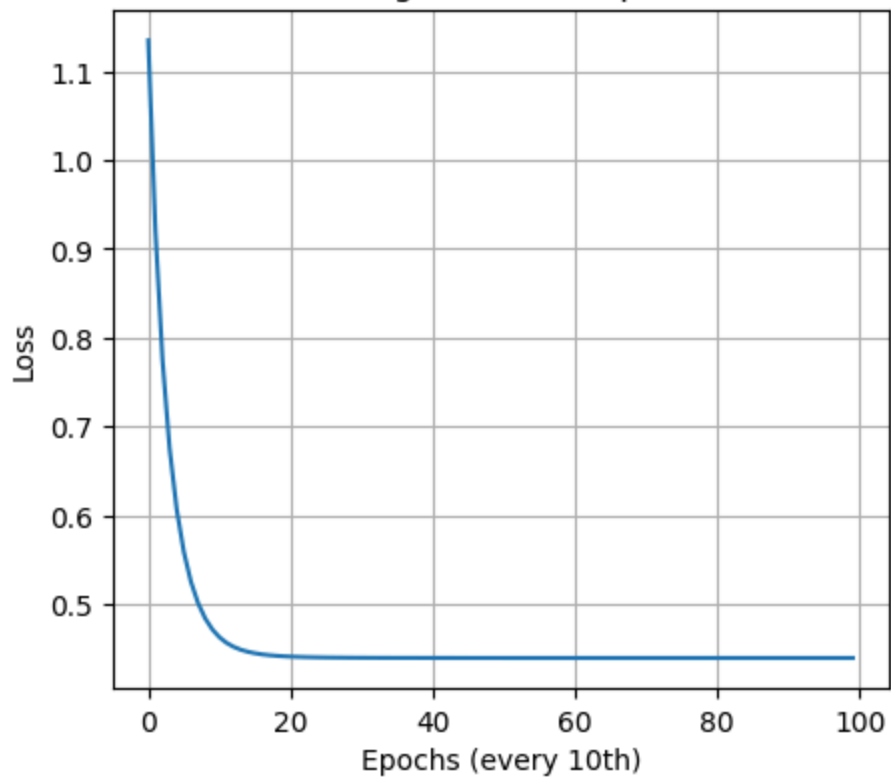
SibSp: -0.2943

Parch: -0.0958

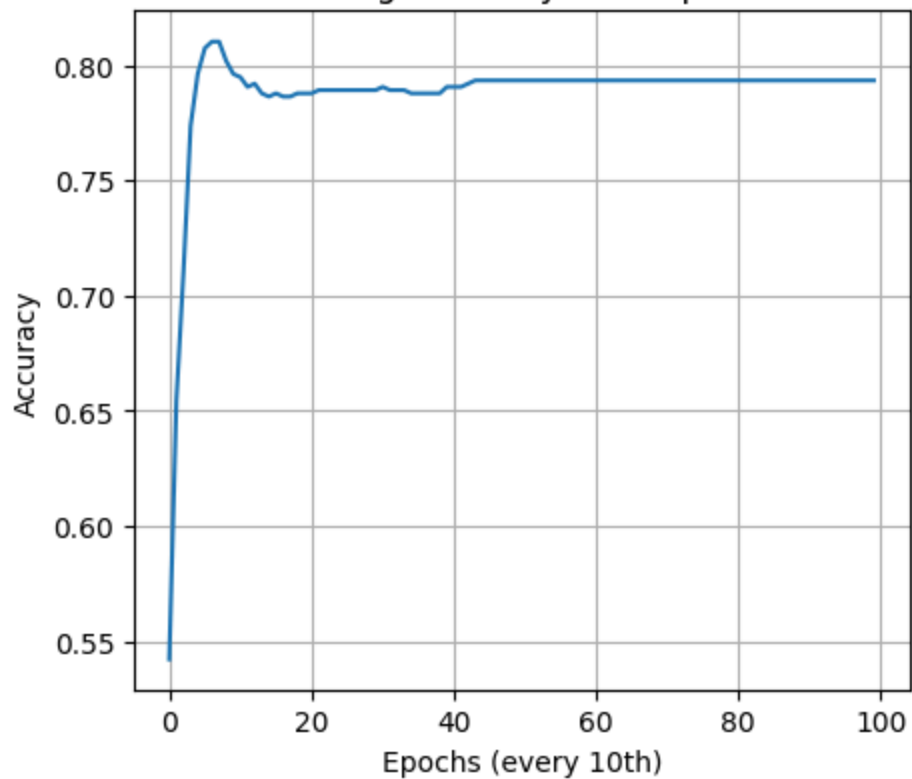
Fare: 0.1394

Bias: -0.6546

Training Loss Over Epochs



Training Accuracy Over Epochs



Training F1 Score Over Epochs

